不是在定时器中断里直接运行协议代码。网络接收还在 workqueue 之上实现了 NAPI 式的预算轮询：中断只调度一次，下半部每轮最多处理 16 个报文，没处理完就重新排队，处理完才重新打开中断。

7.3 控制台、UART 与 TTY

7.3.1 Mini UART

Mini UART 驱动（kernel/uart.c）在 probe() 中完成与早期汇编相同的初始化，再打开接收中断：

```
REGS_AUX->enables |= 1U;                // 使能 AUX 中的 Mini UART
REGS_AUX->mu_control = 0;               // 暂停收发
REGS_AUX->mu_ier = 0;
REGS_AUX->mu_lcr = 3;                   // 8 位数据
REGS_AUX->mu_iir = 0xc6;                // 清空 FIFO
REGS_AUX->mu_baud_rate = 270;           // 115200 @ core_freq 250 MHz
REGS_AUX->mu_ier = MU_IER_RX_ENABLE;    // 接收中断
REGS_AUX->mu_control = 3;               // 打开收发
```

发送使用一个 32 字节的软件环形缓冲区：uartputc() 把字符放进缓冲区，缓冲区满时睡眠；uartstart() 在发送 FIFO 有空位时搬字符过去，并打开“发送空”中断；uartintr() 读空接收 FIFO、把字符交给 consoleintr()，再继续发送。缓冲区空了就关掉发送中断，否则会一直触发。内核 printf 在 panic 等场合使用同步的 uartputc_sync()，不依赖中断。

7.3.2 控制台与行规程

consoleintr() 实现了最基本的行规程：回显、退格、整行缓冲、Ctrl-D 结束输入、Ctrl-P 打印进程表、Ctrl-C 向前台进程组发送 SIGINT。读进程在 consoleread() 里睡眠，直到收到一整行。

7.3.3 TTY 层

/dev/console、/dev/ttyS0、/dev/tty 共用 Mini UART 这一个硬件后端，但含义不同：

- /dev/console 是系统控制台，内核日志直接写 UART，不经过文件系统；
- /dev/ttyS0 是具体的串口终端，init 在上面启动 login；
- /dev/tty 是“当前进程的控制终端”，按调用者的 ctty 动态转发：串口会话转到 ttyS0，SSH 会话转到对应的 PTY 从端。

login 调用 tty_attach() 成为新会话和进程组的组长，并把 ttyS0 设为控制终端；之后 fork 出的 shell 和命令继承会话、进程组和控制终端。shell 运行外部命令时把它设为前台进程组，Ctrl-C 产生的 SIGINT 只发给这个组，所以能终止 ping、tftp 或整条管道，而不会杀掉 shell 和 login。

一个由此暴露的问题是：旧版 ps 通过系统调用让内核 printf 直接打印进程表，结果在 SSH 会话里执行 ps，输出却出现在串口上。内核 printf 写的是系统控制台，不看调用者的 fd 1。现在 sys_ps() 把快照格式化到用户缓冲区，由 ps 自己 write(1, ...)。

7.4 USB 与 HID 键盘

USB 键盘是本项目里协议层次最多的驱动：从主机控制器的寄存器，到 Hub、事务转换器、描述符、HID 报告，再到输入子系统和控制台。本节沿着“插上键盘—识别—配置—读按键”的顺序，把每一层讲清楚：先看硬件路径和三层软件的分工，再看一次事务在总线上如何由 Token、Data、Handshake 三相包组成，然后顺着描述